🔍 Kubernetes Probes (Startup, Liveness, Readiness)

🌟 What are Probes in Kubernetes?

- Probes are used to check the health of containers running inside Pods.
- 👉 Kubernetes uses probes to decide :
  - 🔁 When to restart a container
  - 🚦 When to send traffic
  - ❌ When to stop routing traffic
  - ⏳ Wait for app to start

🎯 Why Probes are Important

- Probes ensure your application:

1. 🟢 Starts properly → Startup Probe
2. 🔄 Restarts if crashed → Liveness Probe
3. 🚦 Receives traffic only when ready → Readiness Probe

🧠 Kubernetes provides 3 types of probes:

🔍 Probe Type	🎯 Purpose	🧠 How It Works	💡 Real-World Use Case
🟢 Startup Probe	🚀 Checks if app has started	👉 Runs during container startup 👉 Blocks other probes until success	🐢 Slow-start apps (Java, heavy services)
❌ Liveness Probe	❤️ App health check	👉 If it fails → container restarts	🔄 Recover from crashes/deadlocks
🚦 Readiness Probe	📡 Checks if app is ready for traffic	👉 If it fails → Pod removed from Service endpoints	⚖️ Avoid sending traffic to unready Pods

🔷 1. Liveness Probe (Alive Check)

- 👉 Checks if the application is `running` properly
- ❌ If fails → Kubernetes `RESTARTS` the container

📌 Example

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
```

🔍 Explanation:

- 🌐 httpGet → Calls → `http://localhost:8080/health`
- ⏱️ initialDelaySeconds → `Waits 5 seconds` before first checking
- 🔁 periodSeconds → Check every `10` seconds
- ❌ If fails multiple times → `container restarts`
- 🎯 Use Case :
  - App stuck
  - Deadlock
  - Not responding

🔷 2. Readiness Probe (Traffic Control)

- 👉 Checks if the container is ready to receive traffic
- ❌ If fails:
  - Pod is removed from Service endpoints
  - No traffic is sent
  - ⚠️ Important:
    - Container still runs
    - Only traffic is `stopped`

📌 Example

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
```

🧠 Explanation

- App may be running but:
  - DB not connected 🗄️
  - Cache not ready ⚡
  - Config not loaded ⚙️

➡️ You DON'T want traffic yet 👉 Traffic is blocked until ready

- 🎯 Use Case :
  - Wait for DB connection
  - Wait for external APIs
  - Warm-up cache

🔷 3. Startup Probe (Slow Start Apps)

- 👉 Used for slow-starting applications (Java, Spring Boot, Large microservices 🚀)
- 👉 What it does:
  - Checks if the application has started successfully
  - ❌ If it fails: ➡️ Kubernetes kills and restarts container

📌 Example

```
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```

🧠 Explanation

- ⏱️ Behavior:
  - ⏳ 30 attempts × 10s = 300s (`5 minutes`) wait
  - During this time: Kubernetes waits before checking other probes
  - ❌ Liveness disabled
  - ❌ Readiness disabled
  - 👉 Gives enough time for app to start

🎯 Use Case

- Apps taking long time to boot
- Heavy initialization

🔁 How All Probes Work Together

- Startup Probe → App `start check`
- Liveness Probe → `Runtime health`
- Readiness Probe → `Traffic control`
- 👉 Best practice: Use ALL three together

🧪 Probe Types

🌐 HTTP Check (httpGet)

```
httpGet:
  path: /health
  port: 8080
```

- 🌍 Calls HTTP endpoint

🔌 TCP Check (tcpSocket)

```
tcpSocket:
  port: 3306
```

- ➡️ Checks if port is open (e.g., MySQL)
- 👉 Verifies container accepts connections on port

🖥️ Command Check (exec)

```
exec:
  command: ["cat","/tmp/healthy"]
```

👉 Executes command inside container

🧠 Key Parameters

🔧 Parameter	📖 Meaning	🧠 How It Works	💡 Real-World Tip
⏱️ initialDelaySeconds	⏳ Delay before first check	👉 Waits before starting probes after container starts	🐢 Increase for slow-start apps
🔁 periodSeconds	🔄 Interval between checks	👉 Defines how often probe runs	⚖️ Balance speed vs overhead
✅ successThreshold	✔️ Success count to be healthy	👉 Number of consecutive successes required	📈 Use >1 for stable readiness
❌ failureThreshold	❗ Fail count to be unhealthy	👉 Number of consecutive failures before action	🛡️ Avoid false restarts
🔍 httpGet / exec / tcpSocket	⚙️ Probe type	👉 Defines how health is checked	🎯 Match app type (API/DB/custom)

🎯 When to Use What?

⚙️ Situation	🔍 Use Probe	🧠 Why This Probe	💡 Real-World Example
💥 App crashes / hangs	❌ Liveness Probe	👉 Detects unhealthy app → restarts container	Node.js stuck loop
⏳ App not ready (DB/cache loading)	🚦 Readiness Probe	👉 Blocks traffic until app is ready	App waiting for DB connection

⚙️ Situation	🔍 Use Probe	🧠 Why This Probe	💡 Real-World Example
🐢 App slow startup	🟢 Startup Probe	👉 Gives time before health checks begin	Java / Spring Boot app startup
🌐 Web server health check	❌ Liveness + 🚦 Readiness	👉 Liveness = ensures app running 👉 Readiness = traffic control	NGINX / API server

🔥 What Happens on Failures?

❌ Probe Type	🚨 Failure Action	🧠 What Kubernetes Does	💡 Real-World Impact
❌ Liveness Probe Failure	🔄 Container is Restart	👉 Kills and restarts the container	Recovers from crashes, deadlocks
🚦 Readiness Probe Failure	🚫 Pod Removed from Service	👉 Stops sending traffic to Pod 👉 Container NOT restarted (Container keeps running)	Prevents errors for users
🟢 Startup Probe Failure	💀 Container Restart	👉 If startup fails → container is killed & restarted	Handles slow or failed initialization

🌍 Real-World Example

💳 Payment Service:

- Before ready, it must:
  - Connect to DB 🗄️
  - Connect to APIs 🌐
  - Load configs ⚙️
  - ➡️ Readiness probe ensures: No traffic allow until everything is ready

---

🧪 Troubleshooting Probes

```
kubectl logs <pod>                                                    #📌 Check Logs
kubectl describe pod <pod>                                            # 📌 Describe
kubectl exec -it <pod> -- curl http://localhost:8080/health             # 📌 Test Ma
```

🔧 Useful kubectl Commands

```
kubectl get pod <pod-name> -o yaml                  # ✅ Check Probe Config
kubectl describe pod <pod-name>                    # ✅ Describe Pod
kubectl delete pod <pod-name>                       # ✅ Delete Pod (Testing Resta
kubectl get events --watch                         # 🔁 Watch Real-Time Events
```

👉 Shows:

- Probe failures
- Pod start/stop
- Scheduling issues

Pod Status

```
kubectl get pods --watch                          # 🔍 Watch Pod Status
```

- 👉 Shows:
  - Pending → Running → Ready
  - CrashLoopBackOff
  - Errors

🔀 Namespace Commands

```
kubectl get events -n my-namespace --watch
kubectl get pods -n my-namespace --watch
kubectl get pods -o wide --watch                    # 🔄 Extra Info
```

🎯 Interview Questions

What happens if readiness probe fails ❓

- 👉 Pod removed from service (no traffic)
- ➡️ No traffic, but container keeps running

What happens if liveness probe fails ❓

- 👉 Container is restarted

Your app takes 90 seconds to start. What will you do ❓

- 👉 Use Startup Probe
  Example:

```
- failureThreshold: 30
- periodSeconds: 3
```

* ➡️ After startup success: Enable Liveness & Readiness

⚠️ Best Practices

- ✅ Always use readiness probe
- ✅ Use startup probe for slow apps
- ✅ Use liveness for health checks
- ❌ Don’t rely only on liveness
- ✅ Combine all 3 probes for production
- ✅ Monitor probe failures regularly

🧾 Final Summary

- 🟢 Startup Probe → Handles slow startup
- ❌ Liveness Probe → Restarts unhealthy apps
- 🚦 Readiness Probe → Controls traffic
- 👉 Together, they ensure:
  - ✔️ High availability
  - ✔️ Zero downtime
  - ✔️ Reliable deployments

🎯 One-Line Answer

Kubernetes probes monitor container health: startup ensures `proper initialization`, `liveness restarts unhealthy containers`, and readiness controls `traffic flow.`